



《计算机科学导论》课程

数据结构与算法

上海交通大学计算机科学导论课程组



致谢：本课件由陈奕廷@CS2021D.SJTU编写；周昕逸@CS2021M.SJTU、宋昱龙@CS2021M.SJTU修改；高晓沅终校定稿

目 录

01

数据结构概述

02

典型数据结构

03

算法基本概念

01.数据结构概述：引言

二十世纪四十年代，为解决弹道学计算问题，电子数字计算机问世。早期应用范围局限于科学与工程计算，处理对象是纯数值性信息，通常称这类问题为**数值计算**。

近几十年来，计算机迅猛发展，性能提升，处理对象从简单的纯数值性信息发展到**非数值性**和**具有一定结构的信息**。如社交网络、基因序列.....

计算机程序处理的一切数值、非数值的信息，乃至程序统称为**数据 (Data)**。

数据表示方法和组织形式直接影响程序处理效率。许多程序规模大、结构复杂、处理非数值性数据，单凭程序设计员经验已难以设计出效率高、可靠性强的程序。

因此，要求人们对计算机程序加工的对象进行系统的研究，即研究**数据的特性以及数据之间存在的关系——数据结构 (Data Structure)**。

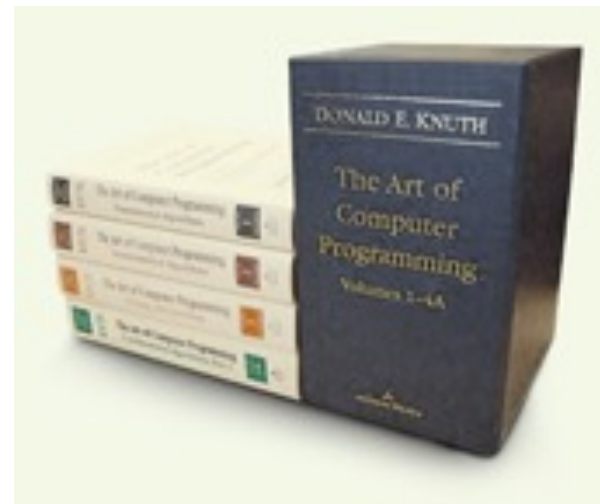
01.数据结构概述：数据结构发展简史与地位

发展史

- 《数据结构》作为一门独立课程从1968年开始设立。
- 1968年美国唐·欧·克努特教授（Donald Knuth）开创了数据结构的最初体系。他所著的《**计算机程序设计技巧**》第一卷《基本算法》是第一本较系统地阐述数据的逻辑结构和存储结构及其操作的著作。

地位

- “数据结构”在计算机科学中是综合性的专业基础课。介于数学、计算机硬件和计算机软件三者之间。
- 不仅是一般程序设计的基础，亦是设计和实现编译程序、操作系统、数据库系统及其他系统程序和大型应用程序的基础。



TAOCP: The Art of Computer Programming

01.数据结构概述：什么是数据结构

计算机解决一个具体问题时，需要经过几个步骤：

- 从具体问题中抽象出一个适当的**数学模型**
- 设计一个解此数学模型的**算法** (Algorithm)
- 编出**程序**、进行测试、调整直至得到最终解答

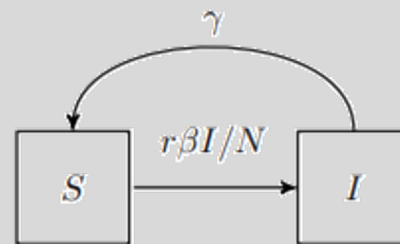
寻求数学模型的实质是分析问题，从中提取操作对象，找出对象之间的关系，并用数学语言加以描述，如：

- 预报人口增长情况的数学模型为微分方程
- 求解梁架结构中应力的数学模型为线性方程组

然而在现实社会中存在着许多非数值计算问题，其数学模型难以用数学方程描述。

例：传染病模型SIS

- **S**USCEPTIBLES: 易感者 (S)，潜在的可感染人群
- **I**NFECTIVES: 感染者 (I)，表现出感染症状的人
- **N**: 总人口数
 β : 易感者得病的概率
 r : 感染者接触到的易感者人数
 γ : 感染者康复或再次患病的概率



$$\begin{aligned}\frac{dS}{dt} &= -r\beta S \frac{I}{N} + \gamma I \\ \frac{dI}{dt} &= r\beta S \frac{I}{N} - \gamma I\end{aligned}$$

01.数据结构概述：非数值计算问题(1)

图书馆的书目检索自动化问题—计算机处理的对象之间存在着线性关系，称为线性的数据结构。

001	高等数学	樊映川	S01	...
002	理论力学	罗远祥	L01	...
003	高等数学	华罗庚	S01	...
004	线性代数	栾汝书	S02	...
...	...	...	...	...

高等数学	001,003,...
理论力学	002,...
线性代数	004,...
...	...

樊映川	001,...
华罗庚	003,...
栾汝书	004,...
...	...

L	002,...
S	001,003,...
...	...

图1.1 图书目录文件示例

01.数据结构概述：非数值计算问题(2)

人机对弈问题—计算机处理的对象是一个个格局。所有可能出现的格局是一棵倒置的树（Tree），表示下一次落子后可能的状态。

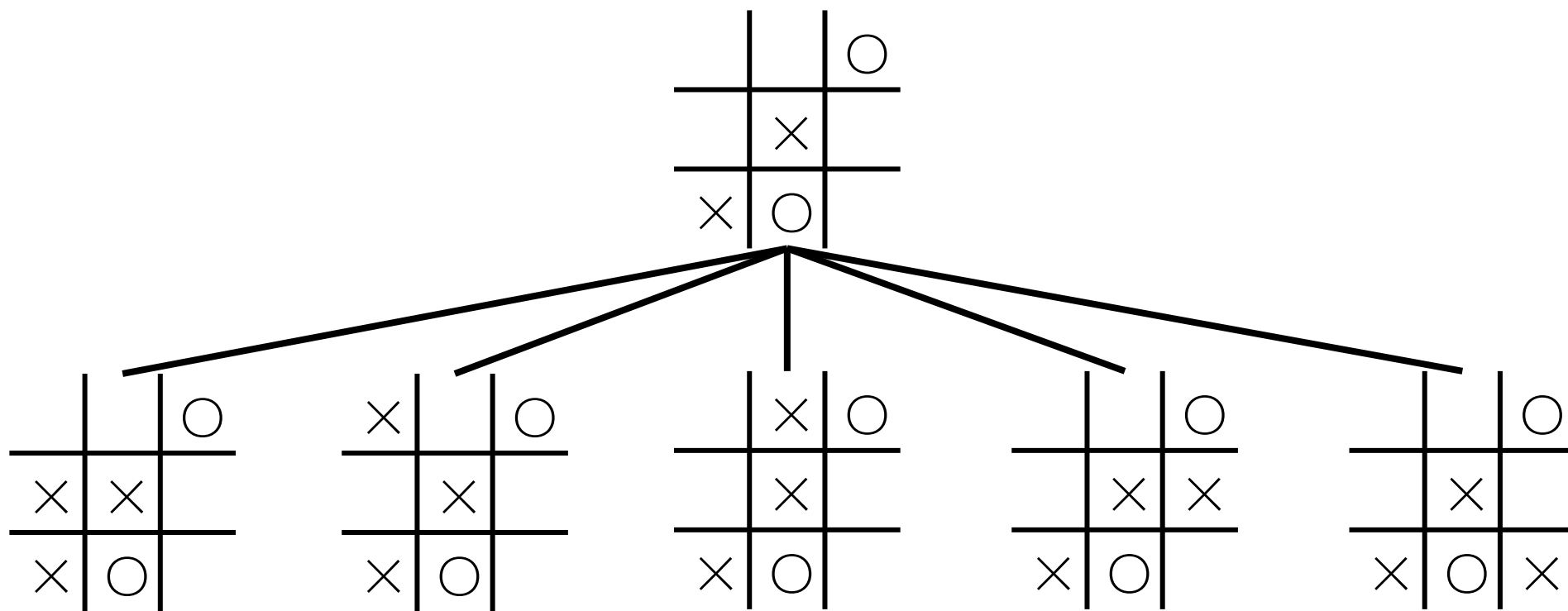


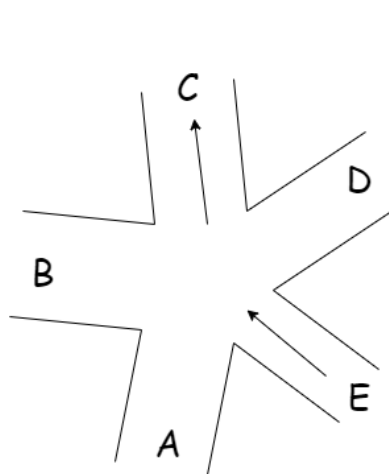
图1.2 棋盘格局示例及其对应的井字棋对弈“树”的局部

多岔路口交通灯的管理问题—数学模型是图（Graph）的数学结构。

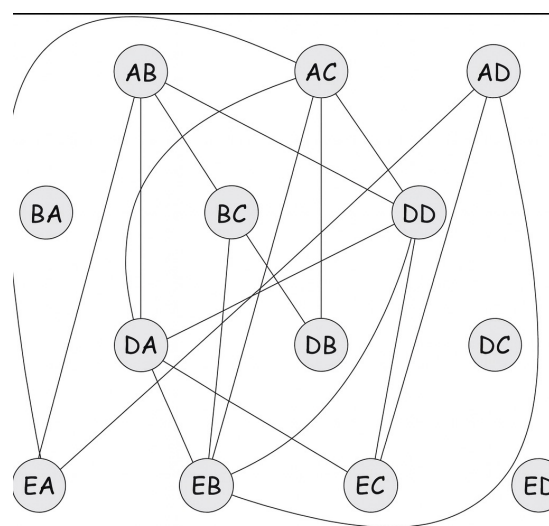
如图所示的五岔路口ABCDE，车辆均靠右行驶，其中C和E为单行道。

右图每个节点代表车的流向，节点之间连边代表这两个流向互相冲突

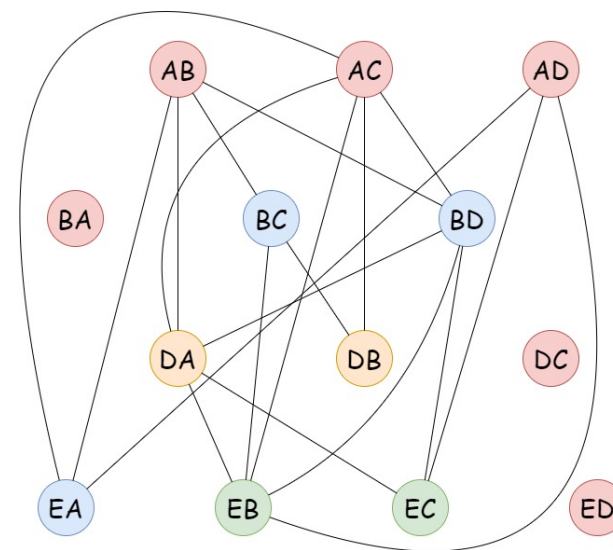
转化为着色问题（邻居节点之间颜色不能相同，目标为找到尽可能多的点颜色相同的方案）



五岔路口



画图建模



着色解决

01.数据结构概述：基本概念和术语（1）

数据(data)：指所有能输入到计算机中并被计算机程序处理的符号的总称。

- 是计算机加工的“原料”
- 文字、表格、图象、声音等都称为数据

数据元素(data element)：组成数据的基本单位。

- 计算机程序中常作为一个整体考虑处理
- 又称为**结点**或**记录**

数据项(data item)：一个数据元素可由多个数据项组成，是数据不可分割的最小单位

- 又称为**字段**或**域**。

学号	姓名	物理	数学	英语
S01012	张强	85	69	90
S01022	李风	77	73	92
S02013	王海	92	91	84

例：学生成绩表

在该学生成绩表中，为了便于处理，把其中的每一行（代表一位学生）作为一个基本单位来考虑，故该数据由3个数据元素构成。

该表中每个**数据元素**都有学号、姓名、语文、数学、英语等五个字段构成。**数据项**是构成数据的最小单位

01.数据结构概述：数据结构的定义

数据结构：是一门研究非数值计算的程序设计问题中计算机操作对象以及它们之间关系和操作的一门学科。

三个要素：对象、关系、操作（运算）

数据对象(data object)：是性质相同的数据元素的集合，是数据的一个子集。

- 例如，一个班级的成绩表，可以看成数据对象。

数据结构(data structure)：

- 定义一：数据元素集合（也可称数据对象）中各元素的关系。
- 定义二：相互之间存在特定关系的数据元素集合。

01.数据结构概述：数据结构的三方面内容

逻辑结构：数据元素之间的逻辑关系。

- 例：在前表的表格数据中，各结点逻辑上存在线性关系，它指出了3个结点在表中的排列顺序。根据这种线性关系，可以看出表中第一位学生是谁，第二位学生是谁，等等。

存储结构(物理结构)：数据元素及其关系在计算机存储器的表示。

- 前表的数据在计算机中可以有多种存储表示，例如，可以表示成数组，存放在内存中；也可以表示成文件，存放在磁盘上，等等。

数据的运算：对数据施加的操作。

- 如对数据进行查找、插入、删除、修改、合并、排序、统计以及简单计算等的操作过程。

算法的设计取决于选定的数据逻辑结构，而算法的实现依赖于采用的存储结构。

为了叙述上的方便和避免产生混淆，通常我们把数据的**逻辑结构**统称为**数据结构 (Data Structure)**，把数据的**物理结构**统称为**存储结构 (Storage Structure)**。

01.数据结构概述：线性表的定义

线性表 (Linear List) 是一种简单的数据结构, 由 n 个“性质相同”的数据元素 ($a_1, a_2, \dots, a_n$) 组成, 这些元素按顺序排列, 像一列火车车厢 (每个车厢只有一个前后邻居)。 $n \geq 0$, 可以为空表。

数据元素的含义广泛, 在不同的具体情况下, 可以有不同的含义:

可以是一个数、一个符号、也可以是一幅图、一页书或更复杂的信息。

例子:

1. 数据序列

1	2	3	4	5	6	7
---	---	---	---	---	---	---

2. 图片序列

						
--	--	--	---	--	--	--

3. 英文字母表 ($A, B, C, \dots, Z$) 是一个长度为26的线性表, 其中的每一个字母就是一个数据元素;

4. 某公司2004年每月产值表 ($400, 420, 500, \dots, 600, 650$) (单位: 万元)是一个长度为12的线性表, 其中的每一个数值就是一个数据元素。

1. 教室里的一排10个座位，已经坐了9人，如果一个迟到的同学想坐在第5个位置（中间），会发生什么？这类似于顺序表的哪个操作？为什么说它“效率低”？
2. 请问你的微信好友列表，在逻辑上属于哪种数据结构？你能想到哪些操作？

01.数据结构概述：线性表的存储方式概述

▪ 两种主要方式：

- **顺序存储（顺序表）**：把元素放在一段连续的存储格子里，像一排紧挨的盒子。

[][][], “连续内存”

- **链式存储（链表）**：把元素分散在内存里，用“指针/链接”把它们按顺序串起来，像用扣子连成的手链。

○→○→○, “指针连接”

▪ 为什么有两种：

不同存储影响操作速度——就像存书：书架（顺序）好快速翻阅，但加书要挪；书签链（链表）易加，但找书要顺链数。选择取决于“你的列表多变还是静态？”

▪ 基本操作：

- 初始化：创建空表
- 插入：在指定位置加元素
- 删除：移除指定元素
- 查找：找元素位置或值
- 存取：检查或更新某个位置的值
- 长度：返回元素个数
- 销毁：释放内存

01.数据结构概述：线性表的特点

线性表中的数据元素类型多种多样，但同一线性表中的元素必定具有相同特性，即属同一数据对象，相邻数据元素之间的关系是线性的，存在着序偶关系。

a_1	a_2		a_{i-1}	a_i	a_{i+1}		a_{n-1}	a_n
-------	-------	-------	-----------	-------	-----------	-------	-----------	-------

- 线性表中元素的个数 n 定义为线性表的长度，为0时(不含任何数据元素的线性表)称为空表。
- a_i 是 a_{i+1} 的直接前驱元素， a_{i+1} 是 a_i 的直接后继元素。第一个元素无前驱，最后一个无后继。
- 元素同质性：同一线性表里的元素具有相同的数据类型或结构。
- 位序：在非空表中的每个数据元素都有一个确定的位置。 a_i 是第 i 个元素，把 i 称为数据元素 a_i 在线性中的位序。
- 线性表可记为 $(a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$

01.数据结构概述：顺序表、访问与定位

- 含义：用一段连续地址保存线性表元素；相邻元素的物理位置也相邻。
- 基地址：第一个元素的存储起始位置，像书架的第一个格子的编号。
- 元素占位一致：每个元素占用的存储空间一样大（如每格都能放一本同样大小的书）。
- 用途：大多数编程语言里，数组即为顺序表的自然载体。
- 随机访问：已知“第 i 个”，可直接跳到对应格子，像拿书架第 i 格的书。
- 为什么能直接跳：每个元素占位一致，相邻元素之间的地址间隔是同一个固定值 L 。
- 连续关系： a_i 与 a_{i-1} 的存储位置相差 L 。
- 公式直观解读：第 i 个元素的存储位置 = 基地址 + $(i-1) \times L$
- 含义总结：知道基地址、知道每格大小、知道第几格，就能一步直达。

地址	1000	1002	1004	1006	1008
存储单元	a	b	c	d	e

01.数据结构概述：顺序表的插入与删除

插入：把x放到第i个位置

- 使用场景：在列表中间“塞”进一个新元素。
- 必要动作：为x腾出空位——从末尾开始，把第n格搬到n+1格，第n-1搬到n格...一直搬到第i格。
- 步骤：
 - 若空间已满，先扩容（另起更大书架，把书整体搬过去）
 - 从尾到头，依次把元素往后挪一格，直到第i格空出来
 - 把x放入第i格，长度加1
- 直觉上的代价：元素越多、插入位置越靠前，需要挪动的越多，操作越慢；在尾部追加最省力。
- 小提示：若允许“末尾追加”，尽量把“常加的内容”放到表尾。

删除：第i个元素

- 使用场景：把中间某个元素拿走并“补齐队形”。
- 必要动作：从第i+1格开始，依次向前挪一格，覆盖掉要删除的那个位置。
- 步骤：
 - 记下或读出第i个元素（若需要）。
 - 从i+1到n，依次把元素向前挪一格。
 - 长度减1（最后一格视为空位）。
- 直觉上的代价：离队尾越近，挪动越少；删表尾最省力。
- 小提示：成批删除时，从后往前删更省力（避免重复搬家）。

01.数据结构概述：顺序表的优缺点与使用场景

优点

- 直接按“第几个”定位，找元素很快、很方便。
- 内存使用紧凑，没有额外链接信息。
- 利于批量计算与缓存友好（连续存放更容易被硬件高效访问）。

缺点

- 在中间插/删，需要成片搬动元素，元素多时会明显变慢。
- 容量一般需要预先估计；不够用时整体扩容并搬家。

适用场景

“读多改少”的列表：成绩册、月份报表、静态字典表。

需要频繁“随机定位”的场景：按编号直接取某元素。

01.数据结构概述：链式存储（链表）

- 含义：用一组“任意位置”的存储单元存放线性表元素，通过“指针”（链接信息）表示元素间的逻辑顺序。
- 核心想法：不要求物理连续，而是用“地址标签”把元素串联起来，像用绳子连珠子。
- 节点结构：每个元素（节点）分成两部分——数据域（存元素值）+指针域（存下一个节点的地址）。
- 直观比喻：火车车厢，每节车厢有乘客（数据）和拉手（指针），车厢可散布轨道，但顺序连好。
- 用途：适合动态变化大的列表，避免顺序存储的“搬家”麻烦。
- 数据域：存放实际元素，如数字“10”或字符串“苹果”。
- 指针域：存下一个节点的“门牌号”（内存地址），像“下一家住址”。
- 头指针：指向第一个节点，像链条的起点（有时加头节点，只存指针不存数据，便于操作）。
- 尾指针（可选）：指向最后一个节点，方便尾部加删。
- 空表：头指针为空（无链条）。
- 示意：头 → [数据: a1 | 指针: 地址2] → [数据: a2 | 指针: 地址3] → [数据: a3 | 指针: null]

01.数据结构概述：链表的类型、访问定位

- 单链表：每个节点只存“下一个”的指针，像单行道，只能从头往前走。
- 双链表：每个节点存“前一个”和“下一个”指针，像双车道，可前后移动。
- 循环链表：最后一个节点指针指向第一个，形成闭环，像圆形跑道，适合循环访问。
- 选择依据：单链表简单省空间；双链表灵活但占多空间。
- 顺序访问：从头指针开始，顺指针一步步走，像牵着手链往前数。
- 为什么不能直接跳：没有固定位置号，必须逐个检查指针（不像顺序表的“格子编号”）。
- 查找元素：从头遍历，直到找到目标或链尾（null）。
- 随机访问难：要找第 i 个，得走 $i-1$ 步指针。
- 含义总结：定位靠“走路”，适合不频繁查的场景；但逻辑顺序清晰。

01.数据结构概述：链表的插入与删除

插入：在p节点后插新节点q

- 使用场景：在任意位置“钩”上新元素，无需搬动别人。
- 必要动作：调整指针，让p钩q，q钩原后继。
- 步骤：
 - 先创建新节点q，存入数据x，q的指针暂为空。
 - q的指针指向p的原后继（q钩原来的下一个）。
 - p的指针指向q（p钩q）。
 - 如果插到尾后，更新尾指针；长度加1。
- 直觉上的代价：找p位置需从头走（步数取决于p在哪），但插本身只需改两个指针，很省力。
- 小提示：头部插入需特殊处理（改头指针）；尾插最简单，像加最后节车厢。

删除：p后的节点q

- 使用场景：从链中“解钩”某个节点，连接前后。
- 必要动作：让p直接钩q的后继，跳过q。
- 步骤：
 - 找到p（q的前一个节点），通过遍历。
 - p的指针指向q的原后继（p钩跳过q）。
 - 释放q节点空间（回收内存）。
 - 如果删尾，更新尾指针；长度减1。
- 直觉上的代价：找p需走链，但删本身只需改一个指针，像拔掉一节车厢重钩。
- 小提示：删头节点，头指针直接指第二个；成批删从头往前，避免断链。

01.数据结构概述：链表的优缺点与使用场景

优点

- 插入/删除只需改指针，不搬动其他元素，变化大时很高效。
- 内存动态分配，不用预估容量，可随时加长链条。
- 存储灵活，元素可散布内存，利用碎片空间。

缺点

- 访问元素必须从头逐个走，找中间或尾部慢（尤其长链）。
- 每个节点多占指针空间（约数据大小的一半），总空间稍多。
- 指针易出错：改错可能“链断”或“循环死”。

适用场景

“改多读少”的动态列表：音乐播放队列（随时加删歌）、浏览器历史（频繁前进后退）。
内存不连续的环境：操作系统任务链表。

01.数据结构概述：顺序表与链表的对比

动画演示参考：<https://visualgo.net/zh>

方面	顺序表	链表
存储	连续盒子	散布链条
访问	直接跳格，快	逐走指针，慢
插/删	大规模搬动，慢	改钩子，快
空间	紧凑	指针额外开销

选择建议：静态多读用顺序；动态多改用链表；编程常混合（如Python list动态数组）。

谢谢！

